

EEN245: Machine Learning with Graphs

Assignment 1

Hüseyin Tepe
Jingxue Yan

February 2, 2026

1 Exercise 1

1.1 Part A

2.

Based on the Karate Club dataset loaded via PyTorch Geometric:

- **Number of nodes:** 34
- **Number of edges:** 156
- **Shape of `data.x`:** $[34, 34]$
- **Shape of `data.edge_index`:** $[2, 156]$
- **Number of classes:** 4

3.

- **`data.x`:** This attribute represents the **node feature matrix**. It has dimensions $[N, D]$, where N is the number of nodes and D is the number of features per node. It contains the raw input attributes for each node used for learning.
- **`data.edge_index`:** This represents the **graph connectivity** in Coordinate Format. It is a matrix of shape $[2, E]$, where the first row contains source node indices and the second row contains target node indices.
- **`data.y`:** This attribute stores the **ground-truth labels** for the nodes. In the Karate Club dataset, it is a vector of shape $[N]$ containing the class ID (0-3) for each member of the club.

1.2 Part B

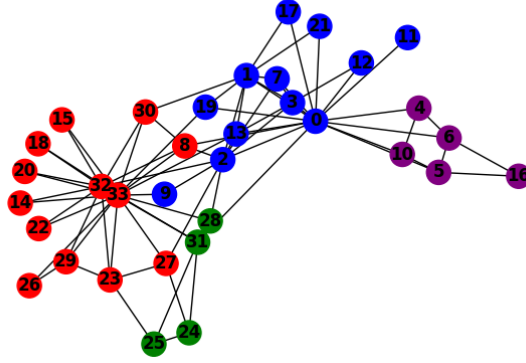


Figure 1: KarateClub nodes and the groups indicated by colors.

1.3 Parts C,D,E

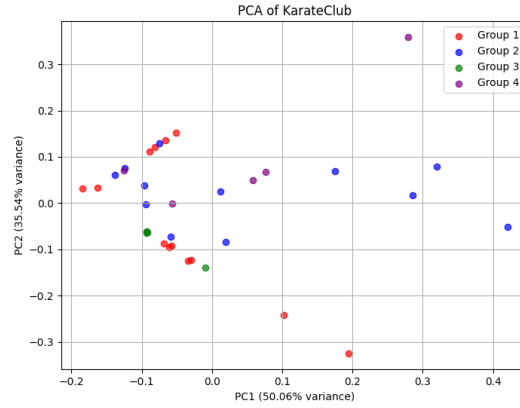


Figure 2: PCA analysis on the karate club nodes.

The data were trained using logistic regression, and a one-vs-rest classifier was used to distinguish the four different groups. Several solvers were tested and the liblinear was the only one able to differentiate the four groups. The other solvers focus on the two largest groups and dismisses the two smaller groups. The liblinear performed better when the class weight setting was set to balanced, as we have two dominant and two smaller groups in our dataset. Table 1 gives the accuracies for different test sizes. Seemingly, the combined embedding matrix gave the best results, followed by the raw data, and lastly Xstruct.

However, data.x does not contain information regarding its surrounding, and as the trained embedding matrix often predicts all to group 1 (red color), where the accuracy is only the ratio of group 1. Although Xstruct seemingly performs worse than data.x, it is differentiating the groups in its output. The division into training and test data performs poorly here, due to the small number of nodes.

Table 1: The accuracy of different embedding matrices and test sizes.

Test size	Xstruct	data.x	Xcomb
0.1	0.25	0.25	0.75
0.2	0.29	0.29	0.57
0.3	0.36	0.45	0.64
0.4	0.29	0.36	0.43
0.5	0.29	0.35	0.35
Test=Train	0.56	1.0	1.0

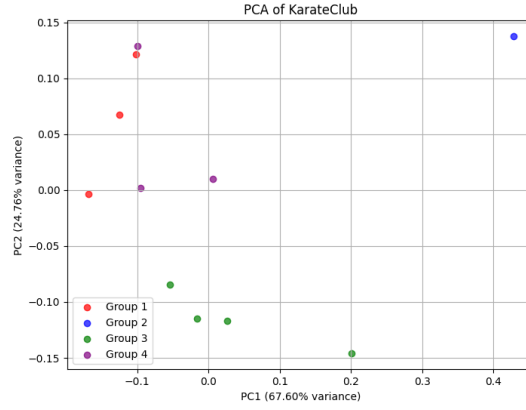


Figure 3: The figure shows the PCA analysis on the testing data, using Xstruct as the embedding matrix. The 34 nodes of the karate club dataset was divided into training and testing data.

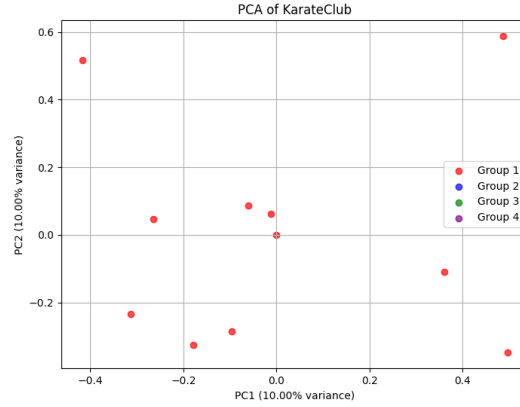


Figure 4: The figure shows the PCA analysis on the testing data, using data.x as the embedding matrix. The 34 nodes of the karate club dataset was divided into training and testing data.

1.4 Part F:

Table 2: Accuracies on the Planetoid (Cora) dataset

Embedding Matrix	Validation Accuracy	Test Accuracy
Xstruct	0.815	0.631
Xattr	0.795	0.611
Xcomb	0.795	0.611

Table 2 tabulates the accuracies for the three embedding matrices. Logistic regression (with standard settings) and a one vs all classifier was used on the training data. The Xstruct matrix, which encodes the citation information, obtained slightly better accuracies. The cora data set contains a citation network, and the node features are binary values of occurring words. The lower accuracy on Xattr may be due to the higher significance of citations, and some unrelated researchers may work on similar topics and use a similar vocabulary. Another possibility may be a lack of dictionary size. The combined matrix did not show any improvement in performance. The reason may be its large size, where the concatenated Xstruct values are effectively suppressed.

1.5 Part G

1. Which input performed best?

Based on the experiments, the **Node Features Only** performed best on the Cora dataset, slightly outperforming the Combined features. On the Karate Club dataset, the Combined features performed best, as the "Features Only" model failed to generalize due to the nature of the one-hot identity features.

2. Why do you think combining them helps or does not help?

Combining features did **not help** for the Cora dataset in our experiment.

- **Reasoning:** The topic of a research paper is primarily determined by its semantic content (keywords), which is fully captured by the node features (X_{attr}).
- **Noise Introduction:** Structural features like Degree or PageRank describe a node's *importance*, not its *topic*. A high-degree node could belong to any class (e.g., a popular AI paper vs. a popular Database paper). Therefore, adding structural features likely introduced noise or redundant dimensionality that confused the simple Logistic Regression classifier.

3. What kind of information is captured by structural features but not by node features?

Structural features capture the **topological role** of a node, which is independent of its content.

- **Centrality (Degree, PageRank):** Identifies influential "hub" nodes.
- **Community (Clustering Coefficient):** Identifies how tightly knit a node's local neighborhood is.
- **Bridges (Betweenness):** Identifies nodes that connect distinct communities.

This information was crucial for the Karate Club (where social factions are defined by connectivity) but less relevant for Cora (where classes are defined by topic keywords).

2 Exercise 2

2.1 Part A:



Figure 5: Original graphs of sets 0, 12 and 20.

2.2 Part B:

- **Parameters Used:** $p = 10$, $q = 0.1$ (BFS-like) and $q = 1$ (DFS-like).
- **Walk Settings:** Walk length $L = 1$, Number of walks $r = 1$.

Node2Vec: BFS-like walk ($q=0.1$)

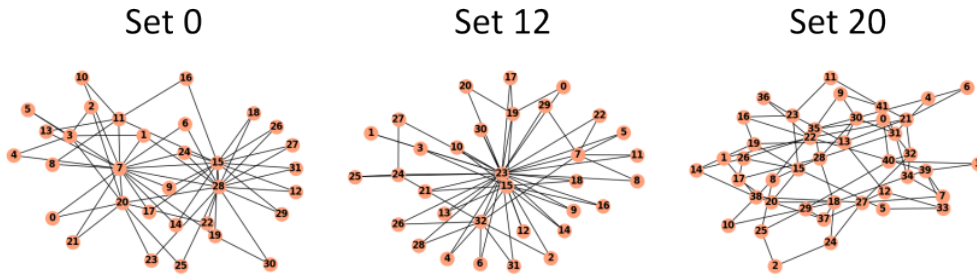


Figure 6: Node2vec graphs ($q=0.1$), using original positional coordinates and edges from node2vec. of sets 0, 12 and 20.

Node2Vec: DFS-like walk ($q=1$)

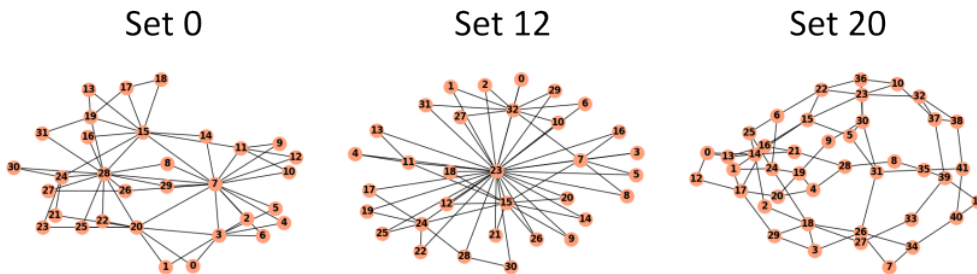


Figure 7: Node2vec graphs ($q=1$), using original positional coordinates and edges from node2vec. of sets 0, 12 and 20.

Figures 8 and 9 use the Z matrix for the positional data.

Node2Vec: BFS-like walk ($q=0.1$)

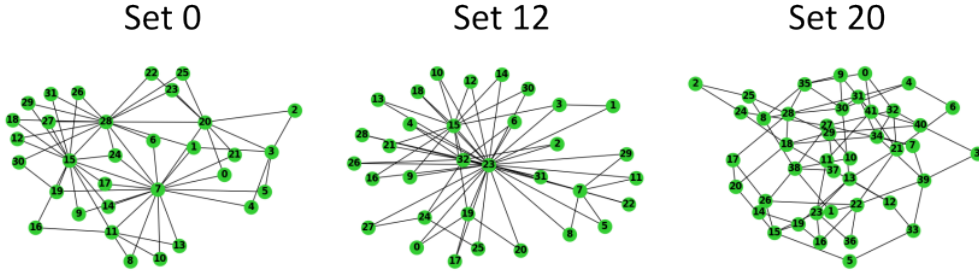


Figure 8: Node2vec graphs, $q=0.1$. Z is used instead of positional data as in figures 6 and 7

Node2Vec: DFS-like walk ($q=1$)

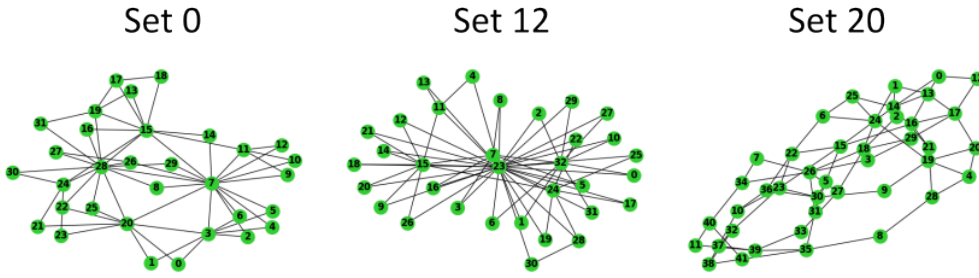


Figure 9: Node2vec graphs, $q=1$. Z is used instead of positional data as in figures 6 and 7

2.3 Part C

Struct2Vec

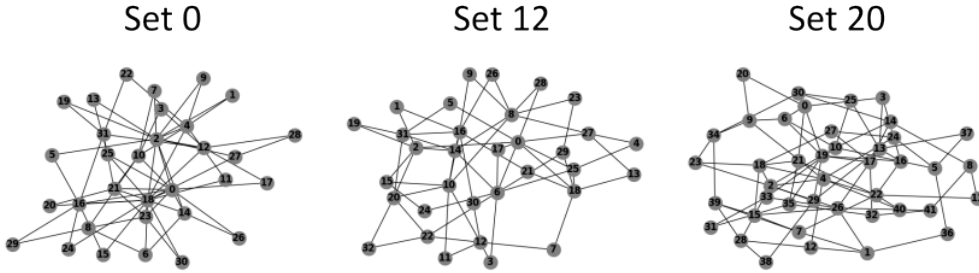


Figure 10: Struct2vec graphs.

2.4 Part D:

1.

Original Geometry Preserves the shape best because the coordinates are explicitly spatial (x, y, z). **Node2Vec** Distorts the shape. It clusters spatially close nodes together but may fold or crumple the mesh because it minimizes distance between connected nodes, losing global geometry. **Struct2Vec** Completely ignores the shape.

2.

Node2Vec mainly captures **proximity-based similarity** (homophily). Nodes that are close to each other in the graph share many common neighbors and appear in the same random walks,

leading to similar embeddings.

3.

Node2Vec relies on random walks that explore the *local neighborhood*. Two nodes that are structurally identical but located far apart will never appear in the same random walk. Since they never co-occur, the model does not learn that they are similar.

4.

Nodes that share **structural roles** but are **distant** in the graph benefit most. Examples include "bridge" nodes connecting communities, "hub" nodes in different parts of a network, or symmetric points on a geometric mesh (e.g., the left and right ears of a horse mesh).

3 Exercise 3

Part A

- **Vocabulary Size:** We limited the graph to the top 3,000 most frequent words. This manages computational complexity while retaining the core semantic vocabulary required for sentiment analysis.
- **Window Size ($w = 2$):** We selected a context window of size 2. This captures immediate neighbors (local context), which is crucial for identifying direct semantic relationships (e.g., "very good") without introducing excessive noise from distant words in long sentences.
- **Edge Weighting:** Edges were weighted by **co-occurrence frequency**. This ensures that words appearing together frequently have a higher probability of being traversed consecutively during the random walks, thereby influencing the embedding proximity.

Part B

B1

Parameter	Value
Embedding Dimension (d)	64
Walk Length (l)	10
Number of Walks (r)	20
Return Parameter (p)	1.0
In-out Parameter (q)	0.5

Table 3: Node2Vec Hyperparameters

Reasoning:

- **Embedding Dimension ($d = 64$):** A dimension of 64 is sufficient to capture the necessary latent semantic distinctions (e.g., separating "positive" adjectives from "negative" ones) without requiring the massive amount of data needed to train high-dimensional vectors.
- **Walk Length ($l = 10$):** A relatively short walk length was chosen to keep the context semantic and local.

- **Node2Vec Parameters** ($p = 1.0, q = 0.5$): This value ensures sufficient coverage of the node’s neighborhood while maintaining computational efficiency.
- **Number of Walks per Node** ($r = 20$): This ensures sufficient statistical coverage of the graph. Since the co-occurrence graph is dense, more walks are needed to accurately approximate the neighborhood distribution.

B2

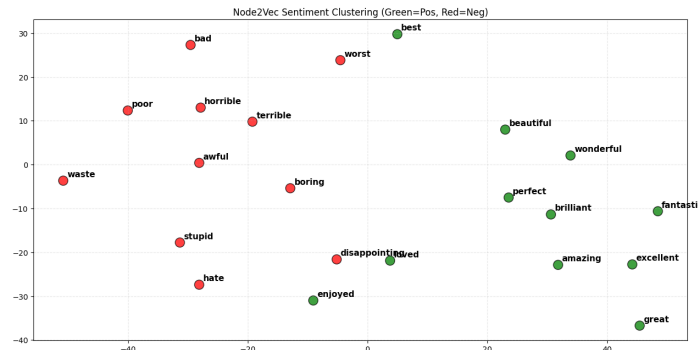


Figure 11: t-SNE visualization of selected word embeddings.

Comment: In the visualization, we observe that words with similar sentiment tendencies cluster together. Positive words (with green color) are clustered on the right while negative words are clustered on the left, demonstrating that the embeddings successfully capture the sentiments of words.

Part C

To represent each document, we computed the **average of the word embeddings** for all words in the document that exist in our vocabulary.

Part D

The dataset was split into training (70%) and testing (30%) sets.

- **Model:** Logistic Regression
- **Test Accuracy: 76%**

Part E

1. The word co-occurrence graph explicitly captures structural relationships and local context, whereas Bag-of-Words treats every word as an isolated, independent feature. Unlike Bag-of-Words, which ignores word order, the graph preserves syntactic proximity by connecting words that appear together and can infer indirect relationships between words that share common neighbors.
2. The parameters p and q dictate whether the random walk explores the local neighborhood or the broader network structure. A low q (BFS-like) keeps walks local to capture homophily, placing words with similar local contexts (synonyms) close together, while a high q (DFS-like) encourages outward exploration to capture structural equivalence, grouping words that play similar functional roles even if they are far apart.

3. The main advantage of this approach is that it produces dense, low-dimensional embeddings that capture latent semantic similarities (e.g., "good" $\approx$ "great"), which sparse methods like TF-IDF fail to recognize. However, this comes at a higher computational cost due to graph construction and random walk generation, and the performance is highly sensitive to the tuning of hyperparameters like window size and walk bias (p, q) compared to simpler, parameter-free methods.

A Source Code for Assignment 1

A.1 Exercise 1

```
1 import torch
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 import matplotlib.colors as mcolors
5 import numpy as np
6 import pandas as pd
7 from torch_geometric.datasets import KarateClub, Planetoid
8 from torch_geometric.loader import DataLoader
9 from torch_geometric.utils import to_networkx
10 import torch_geometric.transforms as T
11 from sklearn.decomposition import PCA
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.multiclass import OneVsRestClassifier
14 from sklearn.model_selection import train_test_split
15 from sklearn.metrics import accuracy_score
16
17 #Part A
18 dataset=KarateClub()
19 loader = DataLoader(dataset,batch_size=20)
20 for row in loader:
21     print(row)
22 #Printed:
23 #DataBatch(x=[34, 34], edge_index=[2, 156], y=[34], train_mask=[34], batch
    =[34], ptr=[2])
24
25 color_labels=['red','blue','green','purple']
26 node_colors=[]
27 for i in range(dataset.x.shape[0]):
28     node_colors.append(color_labels[dataset.y[i]])
29
30 #Part B
31 data_netx = to_networkx(dataset[0], node_attrs=["x"], to_undirected=True)
32 plt.figure(figsize=(6, 4))
33 nx.draw(data_netx, with_labels=True, node_color=node_colors, font_weight='bold
    ')
34
35 def degree(G):
36     #Only for undirected graphs
37     degree_array=torch.zeros(G.x.shape[0],dtype=torch.int32)
38     for i in range(G.x.shape[0]):
39         degree_array[i] = len(G.edge_index[1, G.edge_index[0,:]==i])
40     norm = torch.sqrt(torch.sum(degree_array**2))
41     return degree_array/norm
42
43 def clustering(G):
44     # works for directional and undirectional edges
45     # Number of edges between u and its neighbors divided by the maximum
    possible edges
46     # maximum possible should be 2*(1+2+3+4...) both directions
47     # find the neighbors and count the edge_index vector
```

```

48     clustering_array=torch.zeros(G.x.shape[0],dtype=torch.float32)
49     for i in range(G.x.shape[0]):
50         num_nghbr_u,ind_nghbr_u = neighbors(i,G.edge_index) # outgoing edges
51         for j in ind_nghbr_u:
52             num_nghbr_v = 0
53             _,ind_nghbr_v = neighbors(j,G.edge_index) # outgoing edges
54             for k in ind_nghbr_v:
55                 num_nghbr_v += torch.sum(ind_nghbr_u==k)
56             #check commonality between ind_nghbr_u and ind_nghbr_v
57             # 2: 1
58             # 3: 3
59             # 4: 6
60             # sum(i)from 2 to n
61             den = sum(range(1,num_nghbr_u))
62             # case den = 0 and clustering array becomes infinite
63             if den>0:
64                 clustering_array[i] = (num_nghbr_u + num_nghbr_v)/sum(range(1,
num_nghbr_u))
65             else:
66                 clustering_array[i] = (num_nghbr_u + num_nghbr_v)
67             #neighbors
68         #NORMALIZE
69         norm = torch.sqrt(torch.sum(clustering_array**2))
70         return clustering_array/norm
71
72 def neighbors(u,edge):
73     ind = edge[0,:]==u
74     num_neighbors = torch.sum(ind)
75     ind_neighbors = edge[1,ind]
76     return num_neighbors, ind_neighbors
77
78 def pagerank(G,beta):
79     # Reference http://ilpubs.stanford.edu:8090/422/1/1999-66.pdf
80     # $R(u)=c*\sum(R(v)/N_v+c*(1-\beta))$ 
81     # R: ranks
82     #  $N_v$ : outgoing edges from node v
83     # loop over all nodes for some time until a condition holds
84     tol = 0.01
85     iter = 1
86     iter_max = 100
87     tot_diff_rank = 1
88     R = torch.zeros(G.x.shape[0],dtype=torch.float32)
89     while tot_diff_rank>tol:
90         checkR = R.clone().detach()
91         for i in range(G.x.shape[0]):
92             #count neighbors of u
93             _,ind_nghbr_i = neighbors(i,G.edge_index)
94             for j in ind_nghbr_i:
95                 num_nghbr_j,_ = neighbors(j,G.edge_index)
96                 R[i] = R[j]/num_nghbr_j+(1-beta)
97         tot_diff_rank = torch.sum(torch.abs(checkR-R))
98         if iter>iter_max:
99             print("PageRank: No convergence, max iteration reached!")
100             break
101         # some tot_diff_rank calculation
102     #NORMALIZE
103     norm = torch.sqrt(torch.sum(R**2))
104     return R/norm
105
106 def eigenvector_centrality(G):
107     #  $Ax=\lambda x$ 
108     # A is the adjacency matrix
109     # lambda is the eigenvalue

```

```

110 # x is the score vector, eig_cntrlty here
111 # x = 1/lambda * sum(a*x), a=1 if neighbors, otherwise =0
112 # use: eigenvalues, eigenvectors = torch.linalg.eig(A)
113 # Take the largest eigenvalue and its eigenvector, which is our eig_cntrlty
114 A = torch.zeros(G.x.shape[0],G.x.shape[0])
115 for i in range(G.x.shape[0]):
116     A[i,i] = 1
117     _,ind_nghbr_i = neighbors(i,G.edge_index)
118     for j in ind_nghbr_i:
119         A[i,j] = 1
120 eigenvalues, eigenvectors = torch.linalg.eig(A)
121 max_ind = torch.max(torch.abs(eigenvalues))==torch.abs(eigenvalues)
122 eig_cntrlty = torch.abs(eigenvectors[max_ind])[0]
123 #NORMALIZE
124 norm = torch.sqrt(torch.sum(eig_cntrlty**2))
125 return eig_cntrlty/norm
126
127 def hop_neighbors(G,u):
128     hop_neighbors_list = [torch.tensor([u])]
129     doublettes_neighbors_list = [torch.tensor([u])]
130     all_neighbors = torch.tensor([u])
131     hops=0
132     remaining_nodes = True
133     max_iter = 100
134     while remaining_nodes:
135         k_neighbors = torch.tensor([],dtype=torch.int32)
136         for i in range(len(hop_neighbors_list[hops])):
137             new_index = G.edge_index[1,G.edge_index[0,:]==hop_neighbors_list[
138 hops][i]]
139             for j in new_index:
140                 if torch.sum(all_neighbors==j)==0:
141                     k_neighbors = torch.cat((k_neighbors,torch.tensor([j])))
142                     #all_neighbors = torch.cat((all_neighbors,torch.tensor([j])
143 ))
144             all_neighbors = torch.cat((all_neighbors,k_neighbors))
145             doublettes_neighbors_list.append(k_neighbors)
146             hop_neighbors_list.append(torch.unique(k_neighbors))
147             hops += 1
148             if len(torch.unique(all_neighbors))==G.x.shape[0]:
149                 remaining_nodes = False
150             if hops>max_iter:
151                 print("hop-neighbors: Fail, Max iteration reached!")
152                 break
153     return hop_neighbors_list, doublettes_neighbors_list
154
155 def betweenness(G):
156     # g(v)=sigma_st(v)/sigma_st
157     # sigma_st: total number of shortest paths between s and t
158     # sigma_st(v): shortest number of paths that pass through v
159     # a k-hop algorithm for node v until all nodes are covered for
160     # For 1-hop no nodes are passed
161     # For 2-hop the direct neighbors indices should be added by one
162     # For 2-hop: If two 1-step neighbors share a 2-step neighbor t sigma_st is
163     =2
164     # For 3-hop: 1-hop and 2-hop neighbor indices added by 1
165     # For 3-hop: If 2-step neighbors share common 3-hop neighbors ...
166     # Check if all nodes are covered
167     g = torch.zeros(G.x.shape[0],dtype=torch.float32)
168     sigma_st_v = torch.zeros(G.x.shape[0],dtype=torch.int32)
169     sigma_st = torch.zeros(G.x.shape[0],dtype=torch.int32)
170     g = torch.zeros(G.x.shape[0],dtype=torch.float32)
171     for i in range(G.x.shape[0]):
172         # find all k-hop neighbors

```

```

170         unique_neighbors, doublette_neighbors = hop_neighbors(dataset,i)
171         hops = len(unique_neighbors)
172         if hops>=3: # should not add first and the last of hops
173             for j in range(1,hops-1):
174                 for k in unique_neighbors[j]: # produce the sigma_st(v) vector
175                     sigma_st_v[k] += 1
176                 for k in doublette_neighbors[j]: # produce the sigma_vt
177                     sigma_st[k] += 1
178         for i in range(G.x.shape[0]):
179             g[i] = sigma_st_v[i]/sigma_st[i]
180         #NORMALIZE
181         norm = torch.sqrt(torch.sum(g**2))
182         return g/norm
183
184     def closeness(G):
185         # same as betweenness, but without the nominator
186         g = torch.zeros(G.x.shape[0],dtype=torch.float32)
187         #sigma_st_v = torch.zeros(G.x.shape[0],dtype=torch.int32)
188         sigma_st = torch.zeros(G.x.shape[0],dtype=torch.int32)
189         g = torch.zeros(G.x.shape[0],dtype=torch.float32)
190         for i in range(G.x.shape[0]):
191             # find all k-hop neighbors
192             unique_neighbors, doublette_neighbors = hop_neighbors(dataset,i)
193             hops = len(unique_neighbors)
194             if hops>=2:
195                 for j in range(1,hops):
196                     #for k in unique_neighbors[j]: # produce the sigma_st(v) vector
197                         #sigma_st_v[k] += 1
198                     for k in doublette_neighbors[j]: # produce the sigma_vt
199                         sigma_st[k] += 1
200         for i in range(G.x.shape[0]):
201             g[i] = 1/sigma_st[i]
202         #NORMALIZE
203         norm = torch.sqrt(torch.sum(g**2))
204         return g/norm
205
206     dgr = degree(dataset)
207     btwn_cntrlty = betweenness(dataset)
208     clsnss_cntrlty = closeness(dataset)
209     pr = pagerank(dataset,0.85)
210     eig_cntrlty = eigenvector centrality(dataset)
211     clstr_coeff=clustering(dataset)
212
213     #Part C
214     Xstruct=torch.zeros((dataset.x.shape[0], 6))
215
216     for i in range(dataset.x.shape[0]):
217         Xstruct[i,0] = dgr[i]
218         Xstruct[i,1] = btwn_cntrlty[i]
219         Xstruct[i,2] = clsnss_cntrlty[i]
220         Xstruct[i,3] = pr[i]
221         Xstruct[i,4] = eig_cntrlty[i]
222         Xstruct[i,5] = clstr_coeff[i]
223
224     def plotPCA(X,y):
225         color_labels=['red','blue','green','purple']
226         node_colors=[]
227         for i in range(X.shape[0]):
228             node_colors.append(color_labels[y[i]])
229         # Step 2: Apply PCA (reduce to 2 components for visualization)
230         pca = PCA(n_components=2)
231         X_pca = pca.fit_transform(X)
232         print("Explained variance ratio:", pca.explained_variance_ratio_)

```

```

233     print("Total variance explained:", np.sum(pca.explained_variance_ratio_))
234     df_pca = pd.DataFrame(data=X_pca, columns=['PC1', 'PC2'])
235     df_pca['target'] = y
236     target_names=["Group 1","Group 2","Group 3","Group 4"]
237     plt.figure(figsize=(8, 6))
238     for target, color in zip(range(len(target_names)), color_labels):
239         subset = df_pca[df_pca['target'] == target]
240         plt.scatter(subset['PC1'], subset['PC2'], label=target_names[target],
241                     alpha=0.7, c=color_labels[target])
242     plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.2f}% variance)")
243     plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.2f}% variance)")
244     plt.title("PCA of KarateClub")
245     plt.legend()
246     plt.grid(True)
247 plotPCA(Xstruct, dataset.y,)
248
249 #Multinomial logistic regression
250 # Multi-class classification
251 Xcomb = torch.cat((dataset.x,Xstruct),dim=1)
252 X_train_1, X_test_1, y_train_1, y_test_1 = train_test_split(Xstruct, dataset.y,
253     test_size=0.2, random_state=12)
254 X_train_2, X_test_2, y_train_2, y_test_2 = train_test_split(dataset.x, dataset.
255     y, test_size=0.2, random_state=12)
256 X_train_3, X_test_3, y_train_3, y_test_3 = train_test_split(Xcomb, dataset.y,
257     test_size=0.2, random_state=12)
258
259 #solver='liblinear' best
260 slvr='liblinear'
261 model_1 = LogisticRegression(max_iter=100000,solver=slvr,class_weight='balanced
262     ')
263 multi_model_1 = OneVsRestClassifier(model_1)
264 multi_model_1.fit(X_train_1, y_train_1)
265 print("Multi-class Accuracy:", accuracy_score(y_test_1, multi_model_1.predict(
266     X_test_1)))
267 print(multi_model_1.predict(X_test_1))
268 plotPCA(X_test_1,multi_model_1.predict(X_test_1))
269
270 model_2 = LogisticRegression(max_iter=100000,solver=slvr,class_weight='balanced
271     ')
272 multi_model_2 = OneVsRestClassifier(model_2)
273 multi_model_2.fit(X_train_2, y_train_2)
274 print("Multi-class Accuracy:", accuracy_score(y_test_2, multi_model_2.predict(
275     X_test_2)))
276 print(multi_model_2.predict(X_test_2))
277 plotPCA(X_test_2,multi_model_2.predict(X_test_2))
278
279 model_3 = LogisticRegression(max_iter=100000,solver=slvr,class_weight='balanced
280     ')
281 multi_model_3 = OneVsRestClassifier(model_3)
282 multi_model_3.fit(X_train_3, y_train_3)
283 print("Multi-class Accuracy:", accuracy_score(y_test_3, multi_model_3.predict(
284     X_test_3)))
285 print(multi_model_3.predict(X_test_3))
286 plotPCA(X_test_3,multi_model_3.predict(X_test_3))
287
288 plt.show()

```

Listing 1: Karate

```

1 import torch
2 import torch_geometric
3 import networkx as nx
4 import matplotlib.pyplot as plt

```

```

5 import numpy as np
6 import pandas as pd
7 import torch_geometric.transforms as T
8 from torch_geometric.datasets import Planetoid
9 from torch_geometric.loader import DataLoader
10 from torch_geometric.utils import to_networkx
11 from sklearn.decomposition import PCA
12 from sklearn.linear_model import LogisticRegression
13 from sklearn.multiclass import OneVsRestClassifier
14 from sklearn.model_selection import train_test_split
15 from sklearn.metrics import accuracy_score
16
17 dataset = Planetoid(root='data/Planetoid', name='Cora', transform=T.
    NormalizeFeatures())
18 loader = DataLoader(dataset, batch_size=20)
19 for row in loader:
20     print(row)
21
22 data_netx = to_networkx(dataset[0], node_attrs=["x"], to_undirected=True)
23 # Degree
24 dgr = data_netx.degree()
25 # Betweenness centrality
26 btwn_cntrlty = nx.betweenness centrality(data_netx, normalized=True)
27 # Closeness centrality
28 clsnss_cntrlty = nx.closeness centrality(data_netx)
29 # PageRank
30 pr = nx.pagerank(data_netx, alpha=0.95)
31 # Eigenvector centrality
32 eig_cntrlty = nx.eigenvector centrality(data_netx)
33 # Clustering coefficient
34 clstr_coeff=nx.clustering(data_netx)
35 xrows, xcols = dataset.x.shape
36 Xstruct=torch.zeros((xrows, 6))
37
38 for i in range(dataset.x.shape[0]):
39     Xstruct[i,0] = dgr[i]
40     Xstruct[i,1] = btwn_cntrlty[i]
41     Xstruct[i,2] = clsnss_cntrlty[i]
42     Xstruct[i,3] = pr[i]
43     Xstruct[i,4] = eig_cntrlty[i]
44     Xstruct[i,5] = clstr_coeff[i]
45 for i in range(6):
46     Xstruct[:,i] = Xstruct[:,i]/torch.sqrt(torch.sum(Xstruct[:,i]**2)) #
        normalizing the rows
47
48 Xattr=dataset.x
49 Xcomb = torch.cat((torch.tensor(Xstruct),Xattr),dim=1)
50
51 #For Xstruct
52 model = LogisticRegression(max_iter=100)
53 multi_model = OneVsRestClassifier(model)
54 multi_model.fit(Xstruct, dataset.train_mask)
55 print("For struct:")
56 print("Validation Multi-class Accuracy:", accuracy_score(dataset.val_mask,
    multi_model.predict(Xstruct)))
57 print("Test Multi-class Accuracy:", accuracy_score(dataset.test_mask,
    multi_model.predict(Xstruct)))
58
59 #For Xattr
60 model = LogisticRegression(max_iter=100)
61 multi_model = OneVsRestClassifier(model)
62 multi_model.fit(Xattr, dataset.train_mask)
63 print("For attr:")

```

```

64 print("Validation Multi-class Accuracy:", accuracy_score(dataset.val_mask,
    multi_model.predict(Xattr)))
65 print("Test Multi-class Accuracy:", accuracy_score(dataset.test_mask,
    multi_model.predict(Xattr)))
66
67 #For Xcomb
68 model = LogisticRegression(max_iter=100)
69 multi_model = OneVsRestClassifier(model)
70 multi_model.fit(Xcomb, dataset.train_mask)
71 print("For comb:")
72 print("Validation Multi-class Accuracy:", accuracy_score(dataset.val_mask,
    multi_model.predict(Xcomb)))
73 print("Test Multi-class Accuracy:", accuracy_score(dataset.test_mask,
    multi_model.predict(Xcomb)))

```

Listing 2: Cora

A.2 Exercise 2

```

1 import torch
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 import numpy as np
5 from torch_geometric.datasets import GeometricShapes
6 from torch_geometric.loader import DataLoader
7 from torch_geometric.utils import to_networkx
8 import torch_geometric.transforms as T
9 from torch_geometric.data import Data
10 from torch_geometric.transforms import FaceToEdge
11 import random
12
13 dataset = GeometricShapes(root='data/GeometricShapes', transform=T.
    NormalizeFeatures())
14 loader = DataLoader(dataset, batch_size=20)
15 print(f"Number of graphs: {len(dataset)}")
16 print(f"Feature shape: {dataset.num_features}")
17 print(f"Number of classes: {dataset.num_classes}")
18 # Pick a specific graph by index (e.g., index 0)
19 graph_index = 20
20 data = dataset[graph_index] # This returns a torch_geometric.data.Data object
21
22 # Inspect the graph
23 print("\n=== Graph Info ===")
24 print(data)
25 print(f"Number of nodes: {data.num_nodes}")
26 print(f"Number of edges: {data.num_edges}")
27 print(f"Label: {data.y}")
28
29 dataface = Data(face=data.face)
30 transform = FaceToEdge(remove_faces=False) # Keep faces if needed
31 dataface = transform(dataface)
32 data_edge=Data(x=data.pos[:,0:2], edge_index=dataface.edge_index, y=data.y)
33 data_netx = to_networkx(data_edge, to_undirected=True)
34 plt.figure(figsize=(6, 4))
35 nx.draw(data_netx, with_labels=True, node_color='lightblue', font_weight='bold',
    ')
36 ind=data_edge.edge_index[0,:]==0
37
38 # Node to vec random walk
39 # start: Probabilities 1/q everywhere
40 # save previous point neighbors and compare to new node neighbors
41 # common neighbors have prob=1

```

```

42 # non-common have prob 1/q, and back-step has 1/p
43
44 def node2vec(G,node,p,q,L,iter):
45     all_walked_nodes = np.array([], dtype=np.int32)
46     for i in range(iter):
47         walked_nodes=np.array([], dtype=np.int32)
48         walked_nodes = np.append(walked_nodes,node)
49         # Finding the starting nodes indices (False/True):
50         # data_edge.edge_index[0,:]==walked_nodes[-1]
51         # Getting the neighbors through row nr. 1
52         # data_edge.edge_index[1,data_edge.edge_index[0,:]==walked_nodes[-1]]
53         neighbor = G.edge_index[1,G.edge_index[0,:]==walked_nodes[-1]]
54         weights = np.ones(len(neighbor))*(1/q)
55         for j in range(L):
56             walked_nodes = np.append(walked_nodes,random.choices(neighbor,
weights=weights)) #append next node
57             neighbor_prev = G.edge_index[1,G.edge_index[0,:]==walked_nodes[-2]]
58             neighbor = G.edge_index[1,G.edge_index[0,:]==walked_nodes[-1]] #New
neighbors list
59             weights = np.zeros(len(neighbor)) #New weights vector
60             for k in range(len(neighbor)):
61                 if neighbor[k]==walked_nodes[-2]: #If previous node
62                     weights[k]=1/p
63                 elif any(neighbor[k]==neighbor_prev): # If neighbors are common
64                     weights[k]=1
65                 else: #all others
66                     weights[k]=1/q
67             all_walked_nodes = np.append(all_walked_nodes,walked_nodes)
68     return all_walked_nodes
69 p=10
70 q=1 #q=0.1 for small and q=1 for large
71 L=20 #walk length
72 iter=20
73 X = torch.zeros((data.num_nodes,(L+1)*iter),dtype=torch.int32)
74 for i in range(data.num_nodes):
75     X[i,:] = torch.tensor(node2vec(data_edge,i,p,q,L,iter))
76 d=2
77 # take the d most common:
78 X_d = torch.zeros(X.shape[0],d)
79 for i in range(X.shape[0]):
80     maxes = torch.zeros(X.shape[0],dtype=torch.int32)
81     for j in X[i,:]:
82         if j!=i:
83             maxes[j] += 1
84     sorted_tensor, indices = torch.sort(maxes,descending=True)
85     for k in range(d):
86         X_d[i,k] = indices[k]
87
88 X_edge=torch.zeros(2,data_edge.num_nodes*2)
89 for i in range(data_edge.num_nodes):
90     X_edge[0,2*i] = i
91     X_edge[0,2*i+1] = i
92     X_edge[1,2*i] = X_d[i,0]
93     X_edge[1,2*i+1] = X_d[i,1]
94
95 # B3.2
96 data_node2vec_1=Data(x=data_edge.x,edge_index=X_edge,y=data.y)
97 data_netx_node2vec_1 = to_networkx(data_node2vec_1, to_undirected=True)
98 plt.figure(figsize=(6, 4))
99 nx.draw(data_netx_node2vec_1, with_labels=True, node_color='lightsalmon',
font_weight='bold')
100
101 # B3.3

```



```

102 data_node2vec_2=Data(x=X_d,edge_index=X_edge,y=data.y)
103 data_netx_node2vec_2 = to_networkx(data_node2vec_2, to_undirected=True)
104 plt.figure(figsize=(6, 4))
105 nx.draw(data_netx_node2vec_2, with_labels=True, node_color='limegreen',
        font_weight='bold')
106 plt.show()

```

Listing 3: Node2Vec

```

1 import torch
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 from torch_geometric.datasets import GeometricShapes
5 from torch_geometric.loader import DataLoader
6 from torch_geometric.utils import to_networkx
7 import torch_geometric.transforms as T
8 from torch_geometric.data import Data
9 from torch_geometric.transforms import FaceToEdge
10 import random
11
12 dataset = GeometricShapes(root='data/GeometricShapes', transform=T.
        NormalizeFeatures())
13 loader = DataLoader(dataset,batch_size=20)
14 print(f"Number of graphs: {len(dataset)}")
15 print(f"Feature shape: {dataset.num_features}")
16 print(f"Number of classes: {dataset.num_classes}")
17
18 # Pick a specific graph by index
19 graph_index = 20
20 data = Data(pos=dataset[graph_index].pos,face=dataset[graph_index].face,y=
        dataset[graph_index].y,num_nodes=dataset[graph_index].num_nodes) # This
        returns a torch_geometric.data.Data object
21
22 # Inspect the graph
23 print("\n=== Graph Info ===")
24 print(data)
25 print(f"Number of nodes: {data.num_nodes}")
26 print(f"Number of edges: {data.num_edges}")
27 print(f"Label: {data.y}")
28
29 dataface = Data(face=data.face)
30
31 # Apply FaceToEdge transform
32 transform = FaceToEdge(remove_faces=False) # Keep faces if needed
33 dataface = transform(dataface)
34 data_edge=Data(x=data.pos[:,0:2],edge_index=dataface.edge_index,y=data.y,
        num_nodes=data.num_nodes)
35 data_netx = to_networkx(data_edge, to_undirected=True)
36 plt.figure(figsize=(6, 4))
37 nx.draw(data_netx, with_labels=True, node_color='lightblue', font_weight='bold
        ')
38
39 # struct2vec
40 # find the degrees of all nodes, and put them in some tuple together with
        respective node index
41 # R0(u) is u and s(R0(u)) is the degree of u
42 # R1(u) are the direct neighbors of u, and s(R1(u)) are their nodes
43 # R2(u) should not include R1(u)
44 # weights w(u,v) should give a directional matrix
45 def degree(G):
46     #Only for undirected graphs
47     degree_array=torch.zeros(G.num_nodes, dtype=torch.int32)
48     for i in range(G.num_nodes):
49         degree_array[i] = len(G.edge_index[1, G.edge_index[0,:]==i])

```

```

50     return degree_array
51
52 def dgr_seq(G,u,k):
53     # Return a list of k numpy arrays
54     # use .append() to append to list
55     # s(Rk(u)) gives the k-hop nodes
56     all_nodes = torch.tensor([u],dtype=torch.int32) # Keeping a list of all 0
to k hop neighbor nodes
57     R = [all_nodes] # list of k-hop nodes, first list element is 0-hop, second
1-hop etc...
58     s = [] # s is same as R, but instead of node indices we have their degrees
59     for i in range(k+1):
60         k_neighbors=torch.tensor([],dtype=torch.int32)
61         for j in range(len(R[i])):
62             j_neighbors = G.edge_index[1,torch.tensor(G.edge_index[0,:]==R[i][j
63             ])]
64             k_neighbors = torch.cat((k_neighbors,j_neighbors))
65             k_neighbors = torch.unique(k_neighbors)
66             for j in range(len(all_nodes)):
67                 k_neighbors = k_neighbors[k_neighbors!=all_nodes[j]]
68             all_nodes = torch.cat((all_nodes,k_neighbors))
69             R.append(k_neighbors)
70     # the returned s list needs to contain the degrees
71     dgr = degree(G) # vector of all of the node degrees
72     for i in range(len(R)):
73         len_Ri = len(R[i])
74         dgr_vec = torch.zeros(len_Ri,dtype=torch.int32)
75         for j in range(len_Ri):
76             dgr_vec[j] = dgr[R[i][j]]
77         s.append(dgr_vec)
78     return s
79
80 def dist(su,sv):
81     if len(su)!=0 and len(sv)!=0:
82         dist_max = torch.max(torch.tensor([torch.max(su),torch.max(sv)]))
83         dist_min = torch.min(torch.tensor([torch.min(su),torch.min(sv)]))
84     elif len(su)==0:
85         su = torch.tensor([0])
86         dist_max = torch.max(torch.tensor([torch.max(su),torch.max(sv)]))
87         dist_min = torch.min(torch.tensor([torch.min(su),torch.min(sv)]))
88     elif len(sv)==0:
89         sv = torch.tensor([0])
90         dist_max = torch.max(torch.tensor([torch.max(su),torch.max(sv)]))
91         dist_min = torch.min(torch.tensor([torch.min(su),torch.min(sv)]))
92     distance = dist_max/dist_min - 1
93     return distance
94
95 def struct_dist(G,k):
96     dgr = degree(G)
97     #k=3
98     fk_matrix = torch.zeros((G.num_nodes,G.num_nodes)) # zeros because f-1=0
99     for a in range(G.num_nodes):
100         su = dgr_seq(G,a,k) # will get the degrees of 0 to k hop neighbors of u
101         for b in range(G.num_nodes):
102             if b!=a:
103                 sv = dgr_seq(G,b,k) # will get the degrees of 0 to k hop
neighbors of v
104                 for c in range(k):
105                     # here we find f_k(u,v)
106                     fk_matrix[a,b] = fk_matrix[a,b] + dist(su[c],sv[c])
107                     #g(s(Rk(u)),s(Rk(v)))
108                     #s(Rk(u)) k-neighbor nodes
109                     # dgr_seq(G,u,k) will get 0 to k hop neighbors of u

```

```

109     return fk_matrix
110
111 k=3
112 f = struct_dist(data_edge,k)
113 weights = torch.exp(-f)
114 weights_forWalk = weights
115 for i in range(data_edge.num_nodes):
116     weights_forWalk[i,i] = 0
117     norm = torch.sqrt(torch.sum(weights_forWalk[i,:]**2))
118     weights_forWalk[i,:] = weights_forWalk[i,:]/norm
119
120 Lstruct = 20 # do longer and pick the 2 most common at the end
121 Zstruct = torch.zeros(data_edge.num_nodes,Lstruct,dtype=torch.int32)
122 for i in range(data_edge.num_nodes):
123     for j in range(Lstruct):
124         random_choice = random.choices(torch.tensor(range(data_edge.num_nodes)),
125 ,weights=weights_forWalk[Zstruct[i,j-1],:])[0]
126         Zstruct[i,j] = random_choice
127
128 def most_occurring(Z,d):
129     z_struct = torch.zeros(Z.shape[0],d)
130     for i in range(Z.shape[0]):
131         maxes = torch.zeros(Z.shape[0],dtype=torch.int32)
132         for j in Z[i,:]:
133             if j!=i:
134                 maxes[j] += 1
135         sorted_tensor, indices = torch.sort(maxes,descending=True)
136         for k in range(d):
137             z_struct[i,k] = indices[k]
138     return z_struct
139
140 d=2
141 Zstruct_d = most_occurring(Zstruct,d)
142 struct_edge=torch.zeros(d,data_edge.num_nodes*2)
143 for i in range(data_edge.num_nodes):
144     struct_edge[0,2*i] = i
145     struct_edge[0,2*i+1] = i
146     struct_edge[1,2*i] = Zstruct_d[i,0]
147     struct_edge[1,2*i+1] = Zstruct_d[i,1]
148
149 data_struct2vec=Data(x=Zstruct_d,edge_index=struct_edge,y=data.y)
150 data_struct2vec_netx = to_networkx(data_struct2vec, to_undirected=True)
151 plt.figure(figsize=(6, 4))
152 nx.draw(data_struct2vec_netx, with_labels=True, node_color='gray', font_weight
153         ='bold')
154
155 plt.show()

```

Listing 4: Struct2Vec

A.3 Exercise 3

```

1 import numpy as np
2 import networkx as nx
3 import matplotlib.pyplot as plt
4 import pandas as pd
5 import re
6 import random
7 from collections import Counter
8 from gensim.models import Word2Vec
9 from sklearn.linear_model import LogisticRegression
10 from sklearn.model_selection import train_test_split

```

```

11 from sklearn.metrics import accuracy_score
12 from sklearn.manifold import TSNE
13 import nltk
14 from nltk.corpus import stopwords
15 from sklearn.feature_extraction.text import TfidfVectorizer
16 import kagglehub
17 import os
18
19 try:
20     nltk.data.find('corpora/stopwords')
21 except LookupError:
22     nltk.download('stopwords')
23
24 WINDOW_SIZE_COOCCURRENCE = 2
25 VOCAB_SIZE = 3000
26 EMBEDDING_DIM = 64
27 WALK_LENGTH = 10
28 NUM_WALKS = 20
29 P = 1.0
30 Q = 0.5
31 WINDOW_SIZE_W2V = 5          # Word2Vec window size
32
33
34
35 def preprocess_text(text):
36     """
37     Cleans text: lowercase, remove punctuation, tokenize, remove stopwords.
38     """
39     text = text.lower()
40     text = re.sub(r'[^\w\s]', '', text) # Remove punctuation/special chars
41     tokens = text.split()
42
43     stop_words = set(stopwords.words('english'))
44     tokens = [t for t in tokens if t not in stop_words and len(t) > 2]
45
46     return tokens
47
48 def build_cooccurrence_graph(corpus, window_size=2, vocab_limit=1000):
49     """
50     Constructs an undirected word co-occurrence graph.
51
52     """
53     print("Building vocabulary...")
54     # 1. Build Vocabulary
55     all_words = [word for doc in corpus for word in doc]
56     word_counts = Counter(all_words)
57     common_words = {word for word, count in word_counts.most_common(vocab_limit)}
58
59     print(f"Vocabulary limited to top {vocab_limit} words.")
60
61     # 2. Initialize Graph
62     G = nx.Graph()
63     G.add_nodes_from(common_words)
64
65     print("Adding edges based on co-occurrence...")
66     # 3. Add Edges
67     for doc in corpus:
68         # Filter doc to only include vocab words to speed up and keep graph
        # valid
69         doc_filtered = [w for w in doc if w in common_words]
70
71         for i, target_word in enumerate(doc_filtered):

```

```

72         # Look at words in window (i+1 to i+window_size)
73         # We only look forward because it's undirected
74         end = min(i + 1 + window_size, len(doc_filtered))
75         for j in range(i + 1, end):
76             context_word = doc_filtered[j]
77
78             if target_word == context_word:
79                 continue
80
81             if G.has_edge(target_word, context_word):
82                 G[target_word][context_word]['weight'] += 1
83             else:
84                 G.add_edge(target_word, context_word, weight=1)
85
86     return G
87
88
89 def get_neighbors(G, node):
90     return list(G.neighbors(node))
91
92 def node2vec_walk(G, start_node, walk_length, p, q):
93     """
94     Generates a single biased random walk.
95     Adapted from user provided e1_2.py
96     """
97     walk = [start_node]
98
99     while len(walk) < walk_length:
100         cur = walk[-1]
101         cur_nbrs = get_neighbors(G, cur)
102
103         if len(cur_nbrs) == 0:
104             break
105
106         if len(walk) == 1:
107             # First step: standard weighted random walk
108             # Note: For weighted graphs, we use edge weights
109             weights = [G[cur][nbr]['weight'] for nbr in cur_nbrs]
110             total_w = sum(weights)
111             probs = [w/total_w for w in weights]
112             walk.append(np.random.choice(cur_nbrs, p=probs))
113             continue
114
115         prev = walk[-2]
116
117         # Calculate unnormalized transition probs alpha based on weights
118         weights = []
119         for neighbor in cur_nbrs:
120             edge_weight = G[cur][neighbor]['weight']
121
122             if neighbor == prev:
123                 weights.append(edge_weight * 1/p)
124             elif G.has_edge(neighbor, prev):
125                 weights.append(edge_weight * 1)
126             else:
127                 weights.append(edge_weight * 1/q)
128
129         # Normalize
130         total_w = sum(weights)
131         if total_w == 0:
132             break
133         probs = [w/total_w for w in weights]
134

```

```

135         # Choose next node
136         next_node = np.random.choice(cur_nbrs, p=probs)
137         walk.append(next_node)
138
139     return walk
140
141 def train_node2vec(G, dim, walk_length, num_walks, p, q):
142     """
143     Generates walks and trains embeddings using Skip-Gram.
144
145     """
146     print(f"Generating Node2Vec walks (p={p}, q={q})...")
147     walks = []
148     nodes = list(G.nodes())
149
150     for _ in range(num_walks):
151         random.shuffle(nodes)
152         for node in nodes:
153             walks.append(node2vec_walk(G, node, walk_length, p, q))
154
155     print("Training Word2Vec model...")
156     # Train Skip-Gram (sg=1)
157     model = Word2Vec(sentences=walks, vector_size=dim,
158                     window=WINDOW_SIZE_W2V, min_count=1, sg=1, workers=4,
159                     epochs=5)
160
161     return model
162
163 def get_weighted_document_embedding(doc, model, word2weight):
164     embeddings = []
165     weights = []
166
167     for word in doc:
168         if word in model.wv and word in word2weight:
169             embeddings.append(model.wv[word])
170             weights.append(word2weight[word])
171
172     if len(embeddings) == 0:
173         return np.zeros(model.vector_size)
174
175     # Compute weighted average
176     return np.average(embeddings, axis=0, weights=weights)
177
178 def main():
179     print("Loading data...")
180     path = kagglehub.dataset_download("lakshmi25npathi/imdb-dataset-of-50k-
181     movie-reviews")
182     csv_path = os.path.join(path, "IMDB Dataset.csv")
183
184     # Use 8000 samples for better results
185     df = pd.read_csv(csv_path).sample(8000, random_state=42)
186
187     # Preprocessing
188     print("Preprocessing text...")
189     df['processed_tokens'] = df['review'].apply(preprocess_text)
190
191     # Build Graph
192     print("Building graph...")
193     G = build_cooccurrence_graph(df['processed_tokens'].tolist(),
194                                window_size=WINDOW_SIZE_COOCCURRENCE,
195                                vocab_limit=VOCAB_SIZE)
196     print(f"Graph stats: {G.number_of_nodes()} nodes, {G.number_of_edges()}

```

```

edges")
196
197 # B. Train Node2Vec
198
199 model = train_node2vec(G, EMBEDDING_DIM, WALK_LENGTH, NUM_WALKS, P, Q)
200
201
202 # Visualize Embeddings
203 print("Visualizing Node2Vec embeddings...")
204 # Define symbolic words to check for clustering
205 positive_seeds = [
206     'excellent', 'amazing', 'wonderful', 'fantastic', 'great',
207     'best', 'brilliant', 'loved', 'perfect', 'beautiful', 'enjoyed'
208 ]
209 negative_seeds = [
210     'terrible', 'awful', 'worst', 'horrible', 'bad',
211     'waste', 'boring', 'poor', 'disappointing', 'stupid', 'hate'
212 ]
213
214 # Filter words that actually exist in our trained model
215 selected_words = []
216 selected_colors = []
217
218 for word in positive_seeds:
219     if word in model.wv:
220         selected_words.append(word)
221         selected_colors.append('green') # Positive = Green
222
223 for word in negative_seeds:
224     if word in model.wv:
225         selected_words.append(word)
226         selected_colors.append('red') # Negative = Red
227
228 # If we don't have enough words, fallback to generic top words
229 if len(selected_words) < 5:
230     print("Warning: Specific sentiment words not found in vocab. Plotting
top frequent words instead.")
231     selected_words = list(G.nodes())[:50]
232     selected_colors = ['blue'] * len(selected_words)
233
234 # Prepare vectors
235 vectors = np.array([model.wv[w] for w in selected_words])
236
237 if len(vectors) > 0:
238     # Perplexity must be less than number of samples
239     perp = min(5, len(vectors) - 1)
240
241     tsne = TSNE(n_components=2, random_state=42, perplexity=perp)
242     vectors_2d = tsne.fit_transform(vectors)
243
244     plt.figure(figsize=(10, 8))
245
246     # Plot with specific colors
247     plt.scatter(vectors_2d[:, 0], vectors_2d[:, 1],
248                 c=selected_colors, edgecolors='k', s=150, alpha=0.75)
249
250     for i, label in enumerate(selected_words):
251         plt.annotate(label, (vectors_2d[i, 0], vectors_2d[i, 1]),
252                     fontsize=11, weight='bold',
253                     xytext=(5, 5), textcoords='offset points')
254
255     plt.title("Node2Vec Sentiment Clustering (Green=Pos, Red=Neg)")
256     plt.grid(True, linestyle='--', alpha=0.3)

```

```

257     plt.show()
258
259     print("Training classifier...")
260     # Create X (document embeddings) and y (labels)
261     corpus_strings = [" ".join(doc) for doc in df['processed_tokens']]
262     vectorizer = TfidfVectorizer(vocabulary=list(G.nodes()))
263     tfidf_matrix = vectorizer.fit_transform(corpus_strings)
264     word2weight = dict(zip(vectorizer.get_feature_names_out(), vectorizer.idf_))
265
266     X = np.array([get_weighted_document_embedding(doc, model, word2weight)
267                  for doc in df['processed_tokens']])
268     y = df['sentiment'].map({'positive': 1, 'negative': 0}).values
269
270     # Split
271     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
272                                                         random_state=42)
273
274     # Train
275     clf = LogisticRegression()
276     clf.fit(X_train, y_train)
277
278     # Evaluate
279     preds = clf.predict(X_test)
280     acc = accuracy_score(y_test, preds)
281     print(f"\nSentiment Classification Accuracy: {acc:.4f}")
282
283 if __name__ == "__main__":
284     main()

```

Listing 5: Exercise 3